

I312 - Bases de Datos

Trabajo Práctico Grupal
Registro de experimentos de Machine Learning



Martín Bianchi
Tomás Benavídez
Juan Andrés Quiroga
Franco Amato de Lusarreta

23 de septiembre de 2025

Ingeniería en Inteligencia Artificial

1. Especificación de requerimientos

Escenario y universo conceptual

El escenario seleccionado corresponde al **registro y trazabilidad de experimentos de Machine Learning**. En este dominio, los equipos de trabajo ejecutan múltiples pruebas con distintos datasets, configuraciones de preprocesamiento, hiperparámetros, modelos y métricas de evaluación. Sin un sistema estructurado, se dificulta identificar qué combinación exacta de insumos produjo un determinado resultado, qué modificaciones introdujeron mejoras y cómo reproducir experimentos previos.

El **alcance del modelo** es proveer soporte a un sistema que permita organizar experimentos en proyectos, registrar las ejecuciones (runs) con todos sus elementos asociados, y mantener versiones de datasets y modelos generados. De esta manera, se busca garantizar **reproducibilidad, comparabilidad y trazabilidad** de los resultados.

Requerimientos funcionales

El sistema deberá permitir, al menos, las siguientes funcionalidades:

1. Gestión de proyectos y experimentos

- Crear y administrar proyectos.
- Registrar múltiples experimentos dentro de cada proyecto.
- Asociar metadatos como nombre, descripción, objetivos y fechas.

2. Gestión de usuarios y participación

- Registrar usuarios del sistema.
- Asociar usuarios a proyectos, con un rol y período de participación.
- Restringir la ejecución de runs a usuarios pertenecientes al proyecto.

3. Gestión de datasets y versiones

- Registrar datasets con nombre, descripción y fuente.
- Crear múltiples versiones de un mismo dataset, documentando su preprocesamiento, estrategia de división (train/val/test), semillas y porcentajes de split.
- Mantener la integridad de los vínculos entre datasets y versiones.

4. Gestión de runs (ejecuciones)

- Registrar cada run con número secuencial, estado, tiempos de inicio y fin, usuario responsable y dataset utilizado.
- Permitir que un run genere o no un modelo asociado.
- Asociar a cada run los hiperparámetros empleados y las métricas obtenidas.

5. Gestión de modelos

- Registrar modelos producidos en runs, incluyendo fecha de creación, path de artefacto, framework y algoritmo.
- Mantener relación uno-a-uno entre run y modelo generado.

6. Gestión de hiperparámetros y métricas

- Asociar a cada run un conjunto de hiperparámetros identificados por nombre y valor.
- Registrar métricas de evaluación por nombre, split y valor, asociadas a runs y datasets correspondientes.

7. Consultas y trazabilidad

- Permitir recuperar información sobre proyectos, experimentos y runs en curso o completados.
- Consultar duración de runs, algoritmos utilizados, hiperparámetros más frecuentes, métricas promedio, entre otros.
- Garantizar la trazabilidad completa desde cada métrica hasta el usuario, run, dataset y modelo que la originó.

Diccionario de datos

- **Proyecto:** unidad organizativa que agrupa experimentos con un mismo objetivo.
- **Experimento:** conjunto de runs diseñados para explorar variantes de un mismo problema.
- **Run:** ejecución concreta de un experimento, con usuario, dataset, hiperparámetros y métricas.
- **Dataset / DatasetVersion:** conjunto de datos utilizado en un run; puede tener distintas versiones documentadas.
- **Modelo:** artefacto generado por un run, entrenado con ciertos hiperparámetros y dataset.
- **Hiperparámetro:** configuración empleada en un run, registrada como par (nombre, valor).
- **Métrica:** indicador de desempeño de un modelo, medido en un dataset y split particular.

2. Modelo conceptual

En la Fig. 1 se muestra el diagrama entidad–interrelación que diseñamos. Allí se representan las entidades, sus atributos y las interrelaciones con sus cardinalidades y participaciones correspondientes.

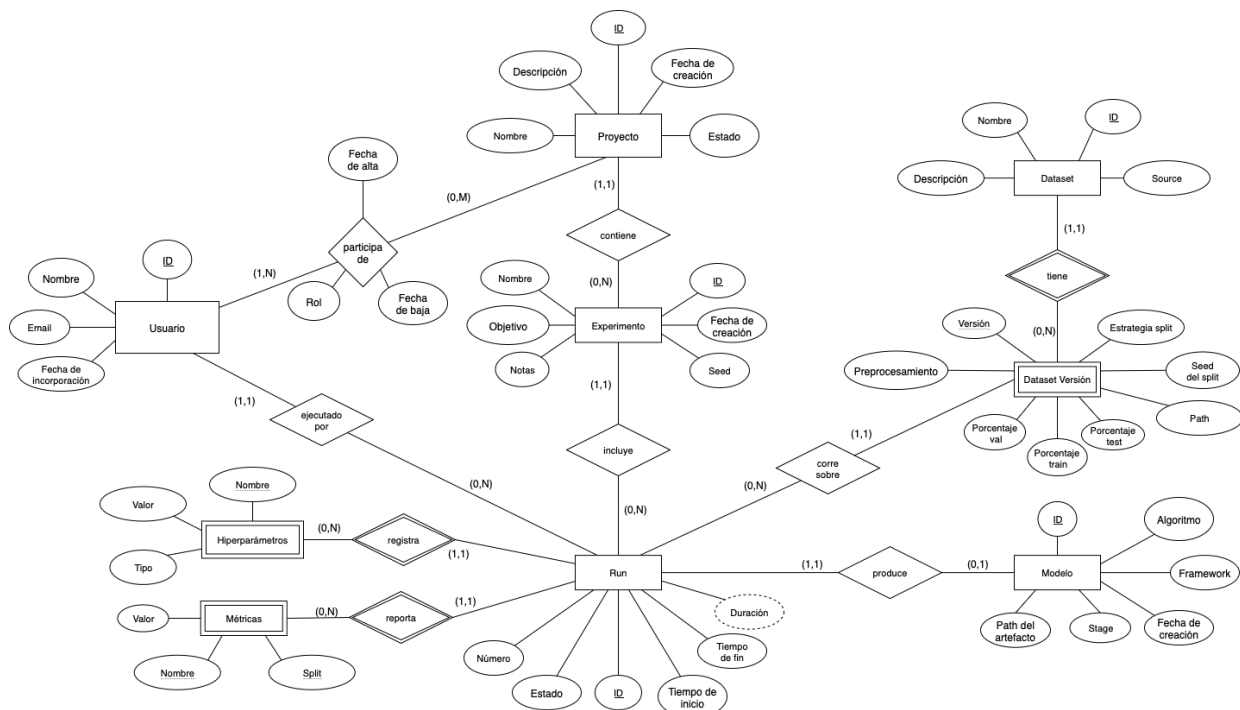


Figura 1: Diagrama entidad–interrelación del sistema de registro de experimentos de ML.

Lectura del modelo y decisiones de diseño

Nuestro punto de partida es que la unidad operativa del dominio es el **run**: una ejecución concreta con parámetros, datos y resultados. Por eso lo modelamos como *nodo central* del diagrama: desde un run puedo reconstruir quién lo ejecutó, en qué experimento y proyecto vive, sobre qué versión de dataset corrió, con qué hiperparámetros se configuró y qué métricas produjo. Esta centralidad simplifica la trazabilidad y, en la práctica, los *joins*.

Proyecto → Experimento → Run (jerarquía). Agrupamos el trabajo en una jerarquía natural: un **Proyecto** contiene varios **Experimentos** (distintas líneas de prueba dentro de un objetivo común) y cada experimento incluye muchos **Runs**. Conceptualmente **Run** es débil respecto de **Experimento** y éste respecto de **Proyecto**; en lugar de propagar claves compuestas, adoptamos **claves subrogadas** (IDs) y expresamos la dependencia con claves foráneas, convirtiendo a estas entidades en fuertes. Esto deja estables las referencias si, por ejemplo, cambia el nombre de un experimento.

Usuarios y participación. No asociamos usuarios “sueltos” a runs: primero modelamos la relación **Participa de** entre **Usuario** y **Proyecto** (con rol y fechas). La decisión es de negocio: *sólo* quien participa del proyecto puede ejecutar runs allí. Luego, cada **Run** queda *ejecutado por* exactamente un usuario responsable. Separar “pertenencia” de “autoría” evita inconsistencias (alguien podría ejecutar sin pertenecer).

Datasets con versión explícita. La reproducibilidad exige capturar *exactamente* la versión de datos usada. Por eso separamos **Dataset** de **DatasetVersion**: cada versión guarda preprocesamiento, estrategia y porcentajes de *split*, semillas y *path*. El run *corre sobre una y sólo una* versión (1:1 desde el lado de run). Alternativas que evaluamos y descartamos: (i) guardar atributos del dataset en **Run** (duplicaba y desincronizaba), (ii) permitir varios datasets por run (complica el TP; si hiciera falta, se modelaría con una N:M adicional).

Modelo como consecuencia opcional. Un run puede fallar o ser puramente evaluativo; por eso **Modelo** es 0:1 respecto de **Run** (cada modelo proviene de un único run, pero no todo run produce modelo). Así, el `id_run` funciona como *clave candidata* de **Modelo** (además del `id_modelo`).

Hiperparámetros y métricas pegados al run. Los **Hiperparámetros** dependen de la ejecución concreta (no del experimento en abstracto ni del modelo final), por eso cuelgan de **Run** y se identifican lógicamente por (`id_run`, `nombre_hp`). Las **Métricas** también cuelgan de **Run** y, para evitar duplicados, usan (`id_run`, `nombre_métrica`, `split`). Una opción alternativa era colgar métricas de **Modelo**; la descartamos porque hay runs sin modelo y porque el dataset utilizado se infiere sin redundancia vía **Run → DatasetVersion**.

Atributos derivados y dominios. La *duración* del run se deriva de `tiempo_fin` – `tiempo_inicio` (por eso aparece punteada); podemos materializarla a nivel físico por eficiencia, pero conceptualmente no es necesaria. En **DatasetVersion** los porcentajes de **train/val/test** viven en $[0, 100]$ y suman 100; el `split` de Métrica pertenece a $\{\text{train, val, test}\}$. Estas decisiones acotan el dominio y previenen estados imposibles.

Claves naturales, candidatas y subrogadas. Reconocemos **claves candidatas** naturales útiles (p.ej., `email` en **Usuario**, `nombre_proyecto` en **Proyecto**, (`id_proyecto`, `nombre_experimento`) en **Experimento**, (`id_dataset`, `versión`) en **DatasetVersion**, (`id_run`, `nombre_hp`) y (`id_run`, `nombre_métrica`, `split`) en tablas dependientes). A nivel lógico-físico decidimos usar **IDs subrogados** como PK por estabilidad y joins más simples, y mantener las candidatas como *restricciones* **UNIQUE NOT NULL** para preservar la unicidad semántica (sin convertirlas en claves primarias).

Por qué esta forma evita problemas. Con estas elecciones evitamos redundancias (no duplicamos configuración de datos ni de splits), eliminamos ambigüedades (siempre hay un `dataset_version` por run) y dejamos allanado el pasaje a FNBC en el modelo lógico: toda dependencia relevante parte de una clave (primaria o candidata) y las entidades dependientes usan claves compuestas mínimas. En síntesis, el diagrama refleja *trazabilidad completa* con el mínimo acoplamiento necesario.

3. Modelo lógico

A partir del diagrama entidad–interrelación construimos el modelo relacional, cuidando que cada entidad y cada interrelación quedaran reflejadas en tablas con sus claves primarias, candidatas y foráneas, como se observa en la Figura 2. Buscamos que las entidades fuertes se conviertan en tablas con su identificador como clave primaria, las entidades débiles incluyan la clave de la fuerte como parte de su clave primaria, y las relaciones se trasladen mediante claves foráneas o, en el caso de muchos a muchos, como tablas intermedias.

En cuanto a los **Experimentos**, se identifican con `id_experimento` y mantienen referencia al proyecto al que pertenecen. La tabla **Run** es central: cada run tiene un número secuencial dentro del experimento, estado, tiempos de inicio y fin, usuario responsable y referencia a la versión de dataset empleada. El vínculo con **Modelo** queda reflejado mediante una clave foránea desde la tabla **Modelo** hacia **Run**, indicando qué run produjo ese modelo. A su vez, los **Hiperparámetros** y **Métricas** se modelan como tablas dependientes del run, cada una con clave compuesta: en el caso de los hiperparámetros, por `id_run` y `nombre_hp`; en el caso de las métricas, por `nombre_métrica`, `split` e `id_run`.

De esta manera, el modelo relacional mantiene la coherencia del conceptual y asegura que cada consulta pueda rastrear exactamente qué usuario, experimento, dataset, versión de dataset, hiperparámetro y modelo están involucrados en cada resultado. Además, al declarar las claves foráneas se refuerza la integridad referencial, garantizando que no puedan cargarse métricas ni hiperparámetros huérfanos sin run asociado, ni runs asociados a experimentos inexistentes.

En varias relaciones, además de la clave primaria se identificaron **claves candidatas** alternativas (por ejemplo, el `email` en la tabla **Usuario**, el `nombre_proyecto` en **Proyecto**, o la pareja (`id_proyecto`, `nombre_experimento`) en **Experimento**). Estas claves candidatas también funcionan como determinantes válidos de los demás atributos, y son relevantes desde el punto de vista lógico ya que imponen restricciones de unicidad adicionales.

Si bien las claves primarias se resolvieron mediante **claves subrogadas** (`id_*`), las claves candidatas se implementaron explícitamente con las restricciones `UNIQUE` y `NOT NULL`. De esta forma se garantiza que los valores de atributos naturales que identifican a las entidades (como `email` o `nombre_proyecto`) no se repitan y que siempre estén presentes, sin asumirlos como claves primarias. Esta decisión trae los beneficios de las claves subrogadas —estabilidad, joins más simples, facilidad de migración— sin perder las garantías de integridad que ofrecen las claves candidatas.

Relación	Clave primaria	Claves candidatas	Claves foráneas
Usuario(id_usuario, nombre_usuario, email, fecha_creacion)	{id_usuario}	{{id_usuario}, {nombre_usuario}, {email}}	-
Dataset(id_dataset, nombre_dataset, source, descripción)	{id_dataset}	{{id_dataset}, {nombre_dataset}}	-
DatasetVersion(versión_dataset, id_dataset, preprocesamiento, seed_split, estrategia_split, pct_train, pct_val, pct_test, path)	{version_dataset, id_dataset}	{{version_dataset, id_dataset}}	{id_dataset} ref. Dataset
Proyecto(id_proyecto, nombre_proyecto, descripción, fecha_creacion, estado)	{id_proyecto}	{{id_proyecto}, {nombre_proyecto}}	-
ParticipaciónEnProyecto(id_usuario, id_proyecto, rol, fecha_alta, fecha_baja)	{id_proyecto, id_usuario}	{{id_proyecto, id_usuario}}	{id_proyecto} ref. Proyecto {id_usuario} ref. Usuario
Experimento(id_experimento, nombre_experimento, id_proyecto, objetivo, notas, fecha_creacion, seed)	{id_experimento}	{{id_experimento}, {id_proyecto, nombre_experimento}}	{id_proyecto} ref. Proyecto
Run(id_run, numero_run, estado, tiempo_inicio, tiempo_fin, id_usuario, version_dataset, id_dataset, id_experimento)	{id_run}	{{id_run}, {id_experimento, numero_run}}	{id_experimento} ref. Experimento {id_usuario} ref. Usuario {version_dataset, id_dataset} ref. DatasetVersion
Modelo(id_modelo, fecha_creacion, path_artefacto, stage, framework, algoritmo, id_run)	{id_modelo}	{{id_modelo}, {id_run}}	{id_run} ref. Run
Hiperparametro(nombre_hp, id_run, valor_json, dtype)	{id_run, nombre_hp}	{{id_run, nombre_hp}}	{id_run} ref. Run
Métrica(nombre_métrica, split, id_run, valor)	{nombre_métrica, split, id_run}	{{nombre_métrica, split, id_run}}	{id_run} ref. Run

Figura 2: Modelo relacional resultante a partir del diagrama entidad–interrelación.

Normalización (FNBC)

Para analizar la forma normal de Boyce–Codd (FNBC) verificamos las dependencias funcionales en cada relación del modelo. Recordemos que una relación se encuentra en FNBC si, para toda dependencia funcional no trivial $X \rightarrow Y$, el conjunto X es una superclave. Por lo tanto, deben considerarse no sólo las claves primarias sino también todas las *claves candidatas* identificadas en el esquema.

- **Usuario(id_usuario, nombre_usuario, email, fecha_creacion)** Claves candidatas: {id_usuario}, {nombre_usuario}, {email}. DFs: cada clave candidata determina los demás atributos. Todas son superclaves. Cumple FNBC.
- **Proyecto(id_proyecto, nombre_proyecto, descripción, fecha_creacion, estado)** Claves candidatas: {id_proyecto}, {nombre_proyecto}. DFs: cada clave candidata determina los demás atributos. Cumple FNBC.
- **Experimento(id_experimento, id_proyecto, nombre_experimento, objetivo, notas, fecha_creacion, seed)** Claves candidatas: {id_experimento}, {id_proyecto, nombre_experimento}. DFs: cualquiera de estas claves determina los demás atributos. Cumple FNBC.
- **Run(id_run, numero_run, estado, tiempo_inicio, tiempo_fin, id_usuario, id_experimento, id_dataset, version_dataset)** Claves candidatas: {id_run}, {id_experimento, numero_run}. DFs: ambas claves determinan el resto de atributos. Cumple FNBC.
- **Modelo(id_modelo, fecha_creacion, path_artefacto, stage, framework, algoritmo, id_run)** Claves candidatas: {id_modelo}, {id_run}. DFs: cualquiera de ellas determina el resto de atributos. Cumple FNBC.
- **Hiperparametro(id_run, nombre_hp, valor_json, dtype)** Claves candidatas: {id_run, nombre_hp}. DFs: esa clave compuesta determina el resto. Cumple FNBC.

- **Metrica(nombre_metrica, split, id_run, valor)** Claves candidatas: {nombre_metrica, split, id_run}. DFs: esa clave compuesta determina el valor. Cumple FNBC.
- **Dataset(id_dataset, nombre_dataset, source, descripcion)** Claves candidatas: {id_dataset}, {nombre_dataset}. DFs: cualquiera de estas claves determina los demás atributos. Cumple FNBC.
- **DatasetVersion(version_dataset, id_dataset, preprocesamiento, seed_split, estrategia_split, pct_train, pct_val, pct_test, path)** Claves candidatas: {version_dataset, id_dataset}. DFs: esta clave compuesta determina el resto. Cumple FNBC.
- **ParticipacionEnProyecto(id_usuario, id_proyecto, rol, fecha_alta, fecha_baja)** Claves candidatas: {id_usuario, id_proyecto}. DFs: esa clave compuesta determina el resto. Cumple FNBC.

En todos los casos, las dependencias funcionales no triviales tienen como determinante una clave candidata (ya sea la primaria o alguna alternativa). No existen dependencias parciales ni transitivas de atributos no primos. Por lo tanto, **todo el esquema se encuentra en FNBC**.

4. Modelo físico

A partir del modelo relacional implementamos el esquema en PostgreSQL mediante un script SQL que crea las tablas, claves primarias, claves foráneas y restricciones necesarias. La decisión general fue utilizar **identificadores internos** (BIGSERIAL) como claves primarias para las entidades principales (Usuario, Dataset, Proyecto, Experimento, Run, Modelo), mientras que en los casos en que el dominio lo exige se adoptaron **claves primarias compuestas**, como en DatasetVersion ((version_dataset, id_dataset)) o en las tablas dependientes Hiperparametro y Metrica.

Las **referencias** respetan fielmente el modelo conceptual:

- Experimento referencia a Proyecto.
- Run referencia a Experimento y al Usuario responsable.
- La versión de dataset empleada por un run se vincula mediante clave foránea compuesta hacia DatasetVersion.
- Modelo mantiene una relación uno-a-uno con Run (sólo si el run produjo un modelo).
- Las tablas dependientes Hiperparametro y Metrica cuelgan de Run, con claves primarias compuestas que evitan duplicados por nombre de hiperparámetro o métrica y split.

Restricciones de integridad y calidad de datos

Se incorporaron restricciones adicionales para reforzar la consistencia:

1. En DatasetVersion se valida que los porcentajes de *split* estén en el rango [0,100] y que su suma sea igual a 100 cuando los tres valores están informados.
2. En Run se controla la coherencia temporal: *tiempo_fin* no puede ser anterior a *tiempo_inicio*.
3. En Experimento se exige que el nombre sea único dentro de cada proyecto (UNIQUE(id_proyecto, nombre_experimento)).

Claves candidatas

Además de las claves primarias (subrogadas con `BIGSERIAL` en la mayoría de las entidades), se reconocieron diversas **claves candidatas** derivadas de atributos naturales. Ejemplos claros son:

- **Usuario:** tanto el `email` como el `nombre_usuario` son claves candidatas.
- **Proyecto:** el `nombre_proyecto` identifica unívocamente proyectos, por lo que se declaró como clave candidata.
- **Dataset:** el `nombre_dataset` es una clave candidata alternativa al `id_dataset`.
- **Experimento:** la pareja (`id_proyecto`, `nombre_experimento`) constituye otra clave candidata además de `id_experimento`.
- **Modelo:** dado que existe una relación uno-a-uno entre `Run` y `Modelo`, el atributo `id_run` también es clave candidata.

La estrategia adoptada fue utilizar **claves subrogadas** como primarias, mientras que las claves candidatas se implementaron mediante restricciones `UNIQUE` y `NOT NULL`. De este modo se garantiza que atributos naturales que deberían identificar instancias (como `email`, `nombre_proyecto` o `nombre_dataset`) sean **únicos y obligatorios**, sin comprometer la estabilidad del esquema en caso de cambios en esos valores.

Políticas de borrado y actualización

Las acciones sobre claves foráneas siguieron el linaje de las entidades:

- `ON DELETE CASCADE` cuando la entidad hija no tiene sentido sin la padre (p. ej., `ParticipacionEnProyecto`, `Hiperparametro`, `Metrica`).
- `ON DELETE RESTRICT` para proteger referencias a `DatasetVersion`.
- `ON DELETE SET NULL` para conservar el registro de `Run` aunque se elimine el `Usuario` que lo ejecutó.

En conjunto, estas decisiones permiten balancear la **estabilidad técnica** del modelo (gracias a las claves subrogadas) con la **integridad semántica** de los datos (mediante claves candidatas implementadas con `UNIQUE NOT NULL`).

5. Carga de datos

Para poblar las tablas del modelo físico con datos ficticios utilizamos la plataforma **Tonic Fabricate**, que permite generar datos sintéticos con estructura relacional, respetando integridad referencial, cardinalidades, y restricciones. Fabricate admite definir un esquema como entrada, generar datos simulados para todos los campos incluyendo tipos específicos (numéricos, texto, fechas, enumeraciones) y exportarlos en formatos compatibles como CSV o SQL, lo que facilita la carga directa en la base de datos PostgreSQL que construimos.

En nuestro caso, generamos instancias para las **10 tablas** del esquema (`Usuario`, `Dataset`, `DatasetVersion`, `Proyecto`, `ParticipacionEnProyecto`, `Experimento`, `Run`, `Modelo`, `Hiperparametro`, `Metrica`). Se cuidó que los datos respeten los constraints del modelo: fecha de finalización de run no anterior a la de inicio, porcentajes de split sumando 100, unicidad de nombre de experimento dentro del proyecto, etc. De esta forma se probó el esquema y se verificó que las consultas funcionaran como se esperaba.

6. Consultas SQL y resultados

A continuación se documentan algunas de las consultas implementadas a modo de ejemplo. Cada consulta se muestra en formato SQL y se acompaña con su salida parcial (primeras 10 filas) capturada desde la terminal de PostgreSQL. El resto de consultas están adjuntadas en formato `.sql`. Para correr todas las consultas una vez cargados los datos en las tablas sólo hay que ejecutar el `run_queries.sh` como se explica más adelante.

```
1 -- duracion_runs_completadas.sql
2 SELECT r.id_run, r.numero_run, r.estado,
3        r.tiempo_inicio, r.tiempo_fin,
4        EXTRACT(EPOCH FROM (r.tiempo_fin - r.tiempo_inicio)) AS duracion_s,
5        e.nombre_experimento,
6        m.algoritmo
7 FROM "Run" r
8 JOIN "Experimento" e ON r.id_experimento = e.id_experimento
9 JOIN "Modelo" m ON m.id_run = r.id_run
10 WHERE r.estado = 'COMPLETED'
11 ORDER BY duracion_s ASC
12 LIMIT 10;
```

Listing 1: Duración de las runs completadas

id_run	numero_run	estado	tiempo_inicio	tiempo_fin	duracion_s	nombre_experimento	algoritmo
1030	4	COMPLETED	2025-04-19 21:21:33	2025-04-19 21:21:33	0.000000	detección-de-fraude-exp-05	Gaussian Mixture
885	8	COMPLETED	2025-04-13 16:10:23	2025-04-13 16:10:27	4.000000	predicción-de-series-tem-exp-04	XGBoost
1424	14	COMPLETED	2025-07-23 18:39:05	2025-07-23 18:39:10	5.000000	predicción-de-demanda-exp-04	
555	14	COMPLETED	2025-07-06 06:42:28	2025-07-06 06:42:35	7.000000	análisis-de-logs-exp-05	Neural Network
187	9	COMPLETED	2025-07-09 19:58:40	2025-07-09 19:58:48	8.000000	análisis-de-sentimiento-exp-05	LDA
701	5	COMPLETED	2025-03-15 22:24:11	2025-03-15 22:24:20	9.000000	clasificación-de-imágene-exp-03	DBSCAN
1149	8	COMPLETED	2025-05-17 17:20:03	2025-05-17 17:20:14	11.000000	segmentación-de-clientes-exp-06	Gradient Boosting
104	1	COMPLETED	2025-01-20 06:37:33	2025-01-20 06:37:44	11.000000	análisis-de-logs-exp-03	CatBoost
600	10	COMPLETED	2025-05-10 10:24:43	2025-05-10 10:24:56	13.000000	modelo-de-traducción-exp-01	
905	10	COMPLETED	2025-07-22 19:52:20	2025-07-22 19:52:36	16.000000	reconocimiento-de-voz-exp-04	Neural Network

Figura 3: Resultados de la consulta de duración de runs completadas (primeras 10 filas).

```
1 -- info_runs.sql
2 SELECT p.nombre_proyecto,
3        e.nombre_experimento,
4        r.id_run,
5        r.numero_run,
6        r.estado,
7        m.framework,
8        m.algoritmo,
9        m.stage,
10       d.nombre_dataset,
11       dv.version_dataset
12 FROM "Run" r
13 JOIN "Experimento" e ON r.id_experimento = e.id_experimento
14 JOIN "Proyecto" p ON e.id_proyecto = p.id_proyecto
15 LEFT JOIN "Modelo" m ON r.id_run = m.id_run
16 LEFT JOIN "DatasetVersion" dv ON r.version_dataset = dv.version_dataset
17                                AND r.id_dataset = dv.id_dataset
18 LEFT JOIN "Dataset" d ON d.id_dataset = dv.id_dataset
19 ORDER BY r.id_run
20 LIMIT 10;
```

Listing 2: Información de los runs

nombre_proyecto	nombre_experimento	id_run	numero_run	estado	framework	algoritmo	stage	nombre_dataset	version_dataset
Análisis de Logs	análisis-de-logs-exp-01	1472	1	FAILED				Fashion-MNIST	3
Análisis de Logs	análisis-de-logs-exp-01	748	2	PENDING	LightGBM	Neural Network	Monitoring	Yelp Reviews	2
Análisis de Logs	análisis-de-logs-exp-01	307	3	RUNNING	TensorFlow	LightGBM	Evaluation	Fashion-MNIST	2
Análisis de Logs	análisis-de-logs-exp-01	634	4	FAILED	PyTorch	Logistic Regression	Deployment	MNIST Digits	4
Análisis de Logs	análisis-de-logs-exp-01	886	5	CANCELLED	LightGBM	Gaussian Mixture	Deployment	Common Crawl Subset	1
Análisis de Logs	análisis-de-logs-exp-01	562	6	PENDING				Heart Disease	1
Análisis de Logs	análisis-de-logs-exp-01	1152	7	RUNNING				Fashion-MNIST	1
Análisis de Logs	análisis-de-logs-exp-01	464	8	RUNNING				Heart Disease	1
Análisis de Logs	análisis-de-logs-exp-01	909	9	PENDING	PyTorch	Random Forest	Development	CIFAR-100	1
Análisis de Logs	análisis-de-logs-exp-01	122	10	PENDING	TensorFlow	K-Nearest Neighbors	Deployment	Fashion-MNIST	1

Figura 4: Resultados de la consulta de información de runs (primeras 10 filas).

```

1  -- mejores_hiperparamsXalgoritmo.sql
2  WITH hyperparameter_performance AS (
3      SELECT m.algoritmo,
4              h.nombre_hp,
5              h.valor_json,
6              mt.nombre_metrica,
7              mt.split,
8              AVG(mt.valor) AS avg_metric
9      FROM "Modelo" m
10     JOIN "Run" r ON m.id_run = r.id_run
11     JOIN "Hiperparametro" h ON r.id_run = h.id_run
12     JOIN "Metrica" mt ON r.id_run = mt.id_run
13     GROUP BY m.algoritmo, h.nombre_hp, h.valor_json, mt.nombre_metrica, mt.split
14 )
15 SELECT *
16 FROM hyperparameter_performance
17 ORDER BY avg_metric DESC
18 LIMIT 10;

```

Listing 3: Mejores hiperparámetros por algoritmo

algoritmo	nombre_metrica	split	nombre_hp	valor_json	avg_metric
CatBoost	accuracy	test	min_samples_split	{"value": 19}	0.9200
CatBoost	accuracy	test	optimizer	{"value": "adam"}	0.9200
CatBoost	accuracy	train	num_epochs	{"value": 48}	0.8200
CatBoost	accuracy	train	optimizer	{"value": "adam"}	0.8400
CatBoost	accuracy	train	min_samples_split	{"value": 19}	0.8400
CatBoost	accuracy	train	num_layers	{"value": 2}	0.9200
CatBoost	accuracy	train	colsample_bytree	{"value": 0.73}	0.9200
CatBoost	accuracy	train	hidden_size	{"value": 16}	0.9600
CatBoost	accuracy	train	loss_function	{"value": "mse"}	0.9600
CatBoost	accuracy	val	loss_function	{"value": "mse"}	0.8800

Figura 5: Resultados de la consulta de mejores hiperparámetros por algoritmo (primeras 10 filas).

En total se implementaron 10 consultas (actividad de usuarios, datasets con más versiones, duración de runs completadas, información de experimentos en proceso, información de proyectos, información de runs, hiperparámetros más usados, mejores hiperparámetros, promedio de métricas y usuarios creados en el último año). Aquí se muestran tres a modo de ejemplo; el resto se encuentran en la carpeta consultas/.

7. Reproducibilidad y estructura del proyecto

La estructura del repositorio debe ser la siguiente:

```
|-- .env
|-- create_tables.sql
|-- initialize_db.sh
|-- run_queries.sh
|-- data/
|   |-- Usuario.csv
|   |-- Dataset.csv
|   |-- DatasetVersion.csv
|   \-- ...
\-- consultas/
    |-- actividad_usuarios.sql
    |-- datasets_mas_versiones.sql
    \-- ...
```

El archivo `.env` contiene las variables de conexión a PostgreSQL:

```
PGHOST=localhost
PGPORT=5440
PGUSER=user
PGDATABASE=mlflow_db
PGPASSWORD=password
```

Es importante chequear que este archivo este en la raíz del directorio con el comando `ls -a` antes de ejecutar los scripts.

El script `initialize_db.sh` lee estas variables, crea la base si no existe y ejecuta `create_tables.sql`. El script `run_queries.sh` recorre la carpeta `consultas/` y ejecuta cada SQL, mostrando la salida en terminal (limitada a 10 filas para facilitar la lectura). Con esto se asegura que cualquier persona pueda reproducir el esquema y las consultas con sólo configurar el `.env`. Todos los `.sql`, `.sh` y `.csv` necesarios para correr el proyecto estan adjuntados con el trabajo. Además hay un `env.txt` que tiene el contenido del `.env` que se utilizó en el trabajo.